CANN开源社区Agent能力介绍

-- 模型、Tilelang、AMCT开发

时间：10月21日

目录

Part 1 CANNBot-Tilelang算子智能体

Part 2 AMCT低精度量化优化智能体

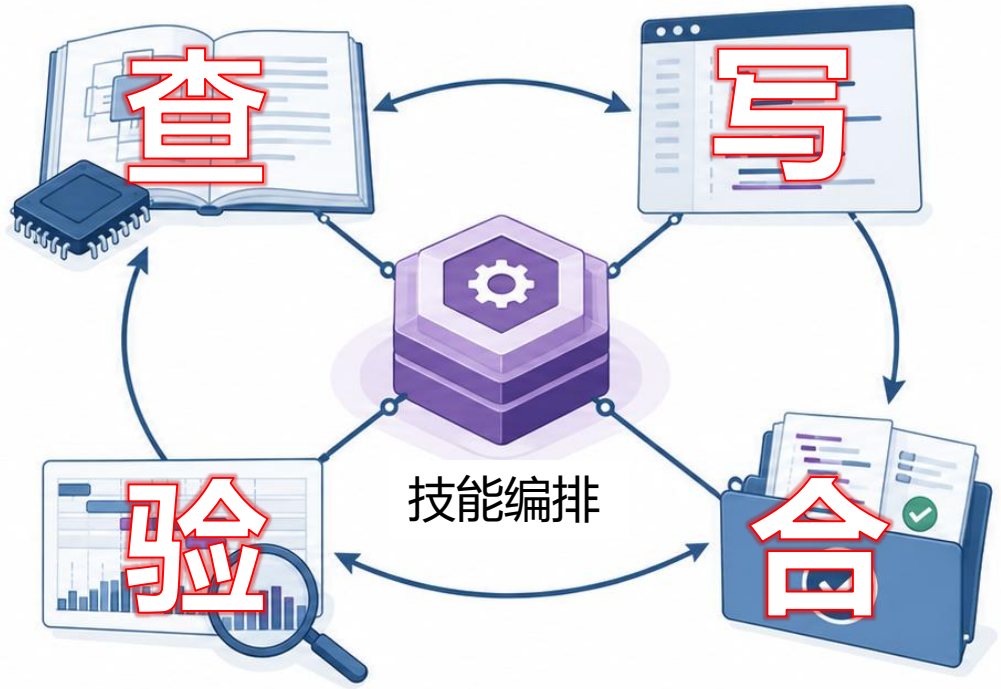
Part 3 CANNBot-模型部署优化智能体

CANNBot-Tilelang算子开发智能体——背景介绍

开发中的痛点



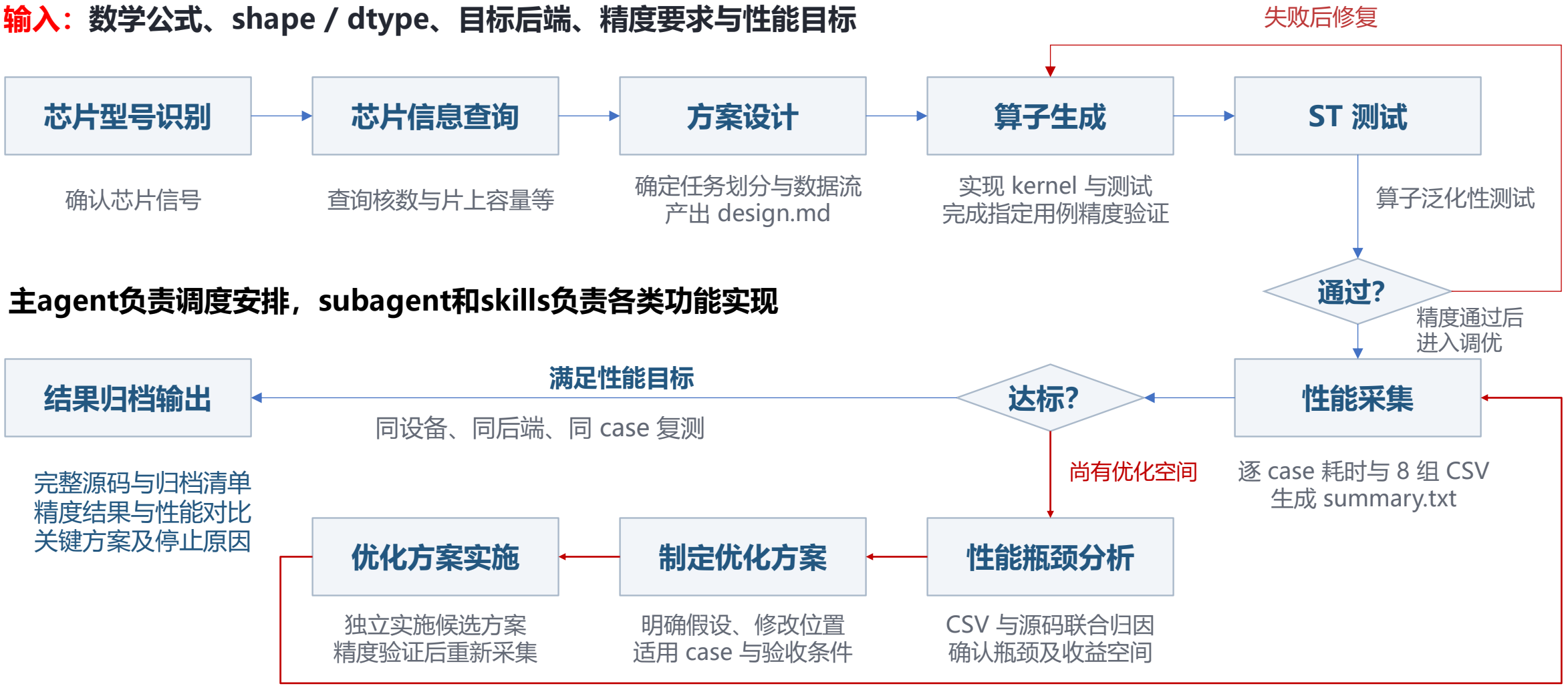
CANNBot-TileLang 的目标



可验证、可复现，并保留可交付结果

CANNBot-Tilelang算子开发智能体——整体工作流

输入：数学公式、shape / dtype、目标后端、精度要求与性能目标

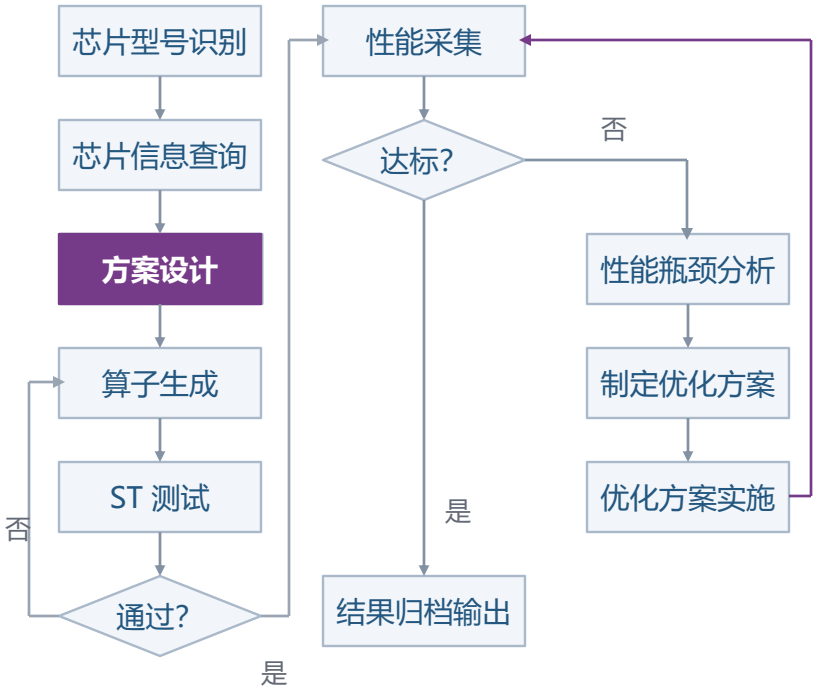


性能优化的同时，保证各case精度通过

CANNBot-Tilelang算子开发智能体——技能介绍

➢ “tilelang-op-design” skills 依据芯片能力和后端实现生成 design.md，为后续代码生成提供参考依据

工作流定位



计算语义决定执行方式

连续轴、归约关系与 dtype 决定 Vector / Cube 路径。
明确输入输出、累加精度及尾块处理，先保证数学语义一致。

数据复用决定搬运方式

按数据依赖安排 GM、UB、L1 / L0 中的存放位置。
中间值尽量在片上消费，同时写清 buffer 生命周期与同步关系。

资源预算决定 tile 大小

预算计入输入输出、中间量、padding 和多版本 buffer。
tile 变大能减少循环，也可能挤占片上空间、降低并行度。

设计交付

任务映射与初始参数、数据流、资源核算、API 依据、验证方法

设计中的参数必须做到查有所依

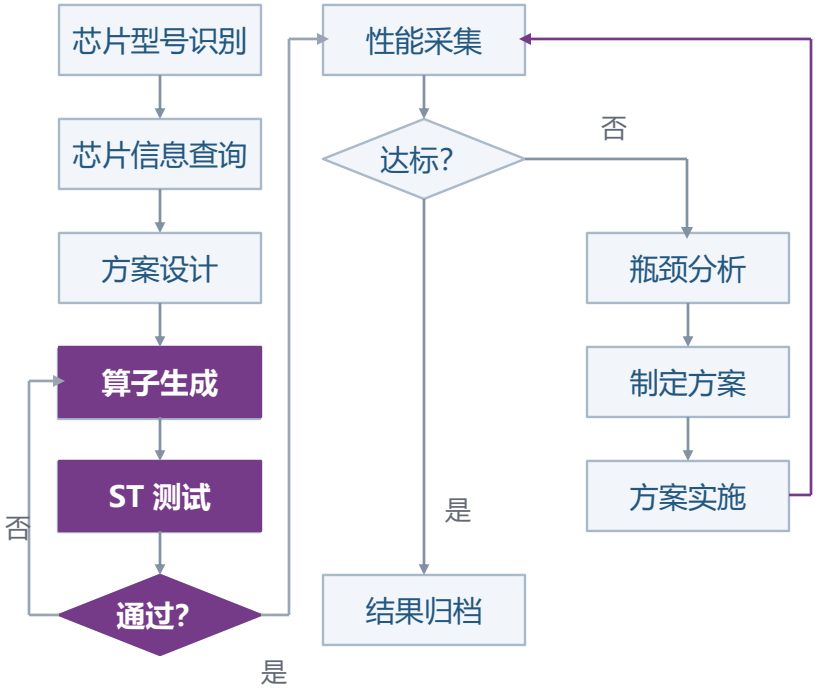
https://gitcode.com/cann

CANN

CANNBot-Tilelang算子开发智能体——技能介绍

- “tilelang-op-develop”skills 落实设计，进行代码开发
- “tilelang-op-test-design”skills 补充系统测试覆盖与执行证据，实现泛化性测试

工作流定位



https://gitcode.com/cann

实现与测试各自独立

<op_name>.py

TileLang kernel 与 JIT 定义
落实索引、初始化、同步及写回

test_<op_name>.py

输入生成、独立 Golden、精度断言
测试实际导入本次交付的 kernel

ST 检查设计边界是否成立

计算与数值	归约轴、广播关系、累加 dtype 与输出转换
形状与访存	整块与尾块、非对齐尺寸、写入范围与越界
执行与覆盖	按测试规范覆盖支持范围，记录实际执行结果

失败后的回路

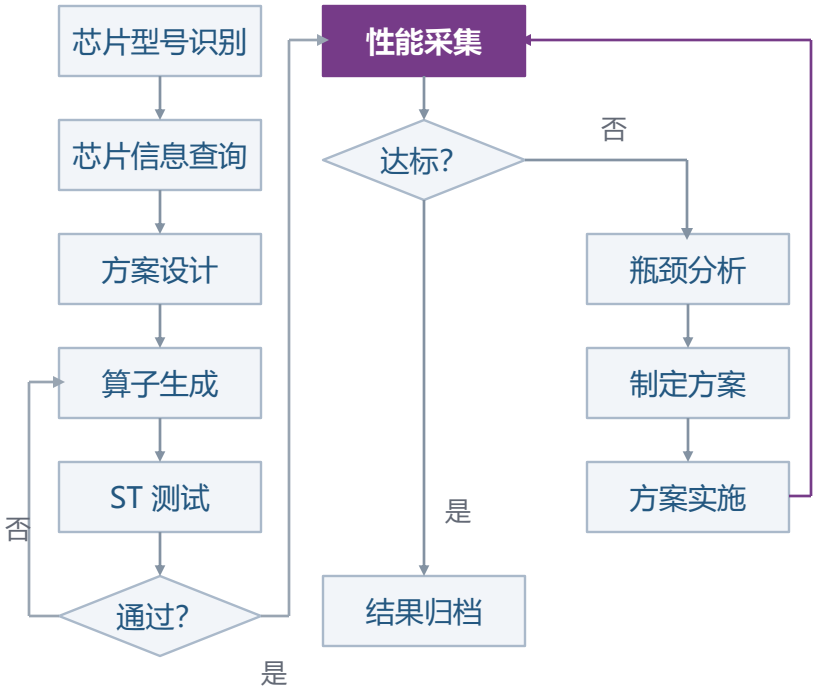
根据报错定位环境、编译、索引、数值或同步问题，
修复后回归相应用例，再进入性能采集。

CANN

CANNBot-Tilelang算子开发智能体——技能介绍

- “tilelang-op-profiling”skills 对待调优的case进行性能采集，保留 8 组原始 CSV 及统计摘要

工作流定位



采集入口与当前源码一一对应

每轮从最新源码重新生成 profiling 入口，按 case 顺序运行。
统一芯片、后端与输入，用 kernel 名过滤辅助计算，避免测到旧实现。

预运行与 JIT → **msprof 采集** → **逐 case 归档** → **summary.txt**

8 组 CSV 保留原始字段，摘要只做目标 kernel 的聚合统计，AIC / AIV 分开。

从耗时建立瓶颈假设

先看总耗时、最长核与主导流水，再从源码核算数据量和计算量。
用原始 CSV、时间线或生成代码确认等待、重复工作与核间不均衡。

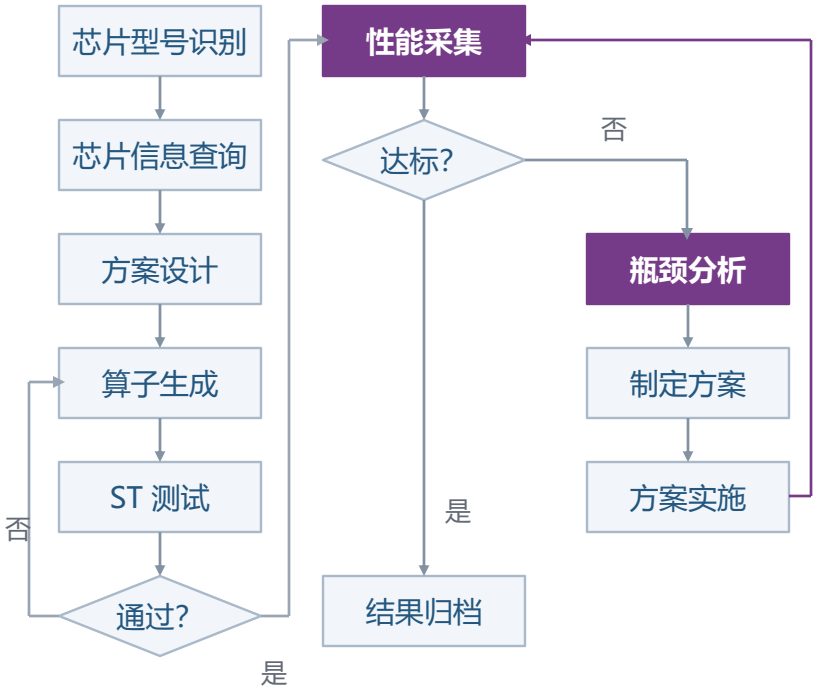
搬运下界 ≈ 实际搬运字节 / 对应带宽

计算下界 ≈ 实际运算量 / 对应单元算力

总耗时、核内耗时、各流水占比口径不同；AIC / AIV 时间不能直接相加

CANNBot-Tilelang算子开发智能体——技能介绍

工作流定位



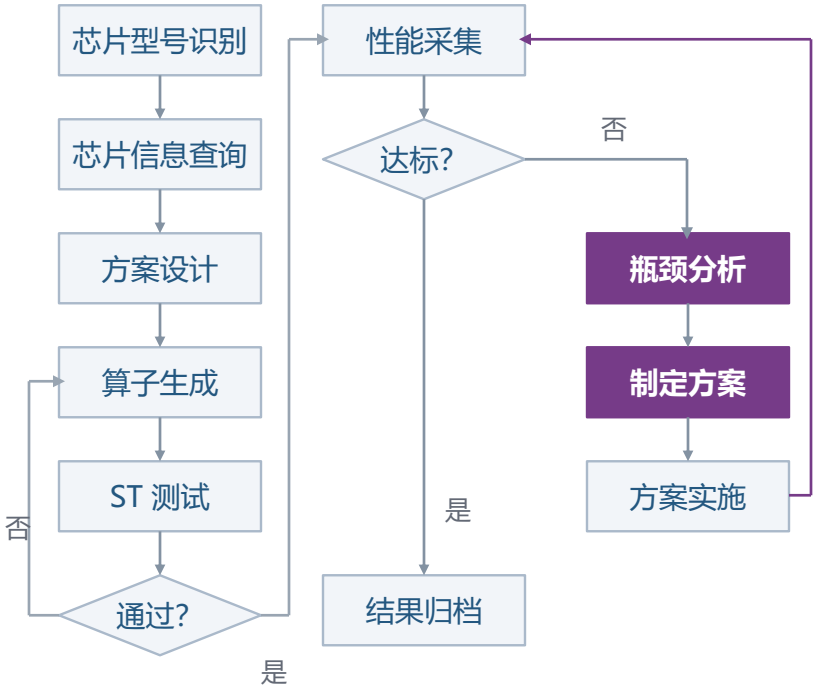
指标文件（.csv）	分析指标	分析作用
OpBasicInfo	耗时、Block Dim、运行频率	确认目标与测量条件
PipeUtilization	逐核时间、各流水占比	定位主导流水与负载尾差
ArithmeticUtilization	指令类型、条数与运算量	发现重复计算、Cast 开销
Memory	搬运字节、带宽、指令数	区分带宽压力与零碎搬运
MemoryL0	L0A / L0B / L0C 读写	检查 GEMM 数据供给与复用
MemoryUB	Vector / Scalar 对 UB 的读写	检查片上反复读写与标量访问
L2Cache	读写命中次数与命中率	验证访问顺序和数据局部性
ResourceConflictRatio	bank / bank group 冲突、等待	定位布局或资源依赖问题

CANN

CANNBot-Tilelang算子开发智能体——技能介绍

- “tilelang-perf-optimization”skills 形成可验证方案，
- “tilelang-performance-best-practices”skills 提供实现参考

工作流定位



瓶颈现象	优先尝试的手段	复测时看什么
搬运主导 MTE2 / MTE3 耗时高	连续搬运、对齐与合并小 copy 复用输入，减少 GM 中间结果	搬运字节或指令数下降 总耗时是否同步下降
计算主导 VEC / CUBE 耗时高	融合重复遍历，减少无效 Cast 调整 GEMM tile 与 L1 / L0 复用	有效工作量与指令数 精度、总耗时和片上占用
标量或固定开销大 小 shape 尤其明显	外提索引计算，简化动态分支 合并小任务，按规模调整核数	Scalar 绝对耗时 不同 shape 的收益与退化
并行度不足或尾差大 各核完成时间差异大	调整切分轴、tile 与任务映射 平衡任务数量和单核工作量	最慢核耗时与尾块 任务覆盖是否完整
流水串行或片上冲突 等待多、重叠不足	检查 stage 与 buffer 版本 调整 stride / padding / 生命周期	时间线重叠与冲突占比 同步正确性和资源容量

例如：MTE2 高但带宽低，先查粒度和对齐；接近带宽上限，再查复用与流水

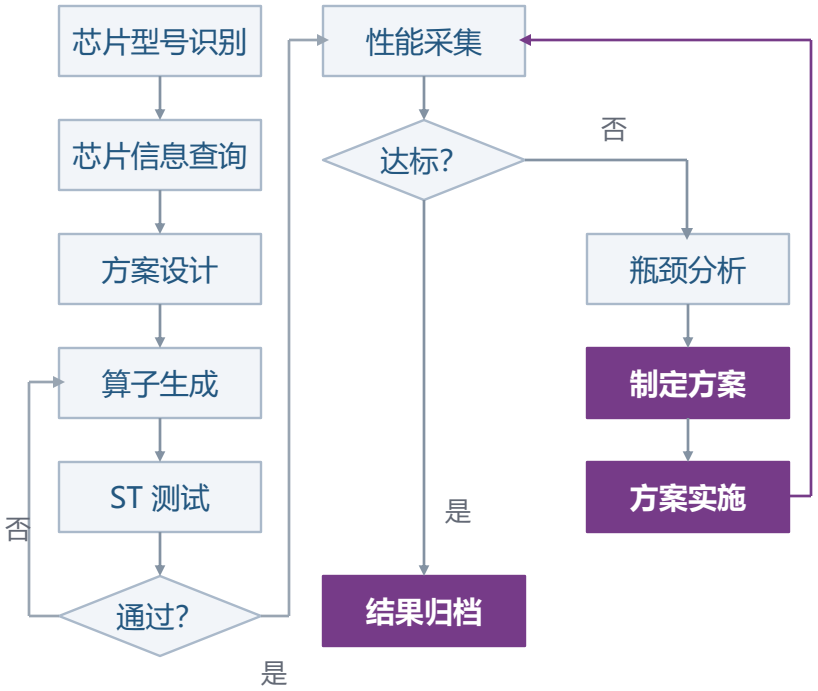
https://gitcode.com/cann

CANN

CANNBot-Tilelang算子开发智能体——技能介绍

分析阶段交付可实施方案，实施阶段验证精度与性能，保留无收益方案的原因

工作流定位



方案写到可以直接验证

记录瓶颈证据、修改位置、适用 case、预期变化和回退条件。
有 API 与 lowering 依据的方案进入实施，仅有思路的候选继续保留。

方案独立实施，基线始终可回退

修改实现 → 精度验证 → 同口径采集 → **逐 case 对比**

逐 case 加速比 = 基线耗时 / 优化后耗时

汇总报告给出几何平均，同时列出最差和退化 case。
小幅收益接近测量波动时交替复测，不凭一次结果替换基线。

最终交付保留结果，也保留依据

完整源码与归档清单、精度证据、全部 case 性能对比、
关键优化方案和停止原因。达到目标、仍未达标或无收益分别说明。

CANN

CANNBot-Tilelang算子开发智能体——效果展示

使用方法

① git clone https://gitcode.com/cann/cannbot-skills.git

② 进入 plugins-official —— tilelang-op-orchestrator 路径

③ 执行 bash init.sh --tool opencode

④ 终端输入 opencode

⑤ 输入算子开发指令，会自动触发工作流，如：

帮我开发xxx算子，支持输入xxx类型的shape和dtype。

基于CANNBot-Tilelang算子开发完整工作流

25 个算子

155 个 case

100% case 精度通过

>80% 带宽利用率

CANN

目录

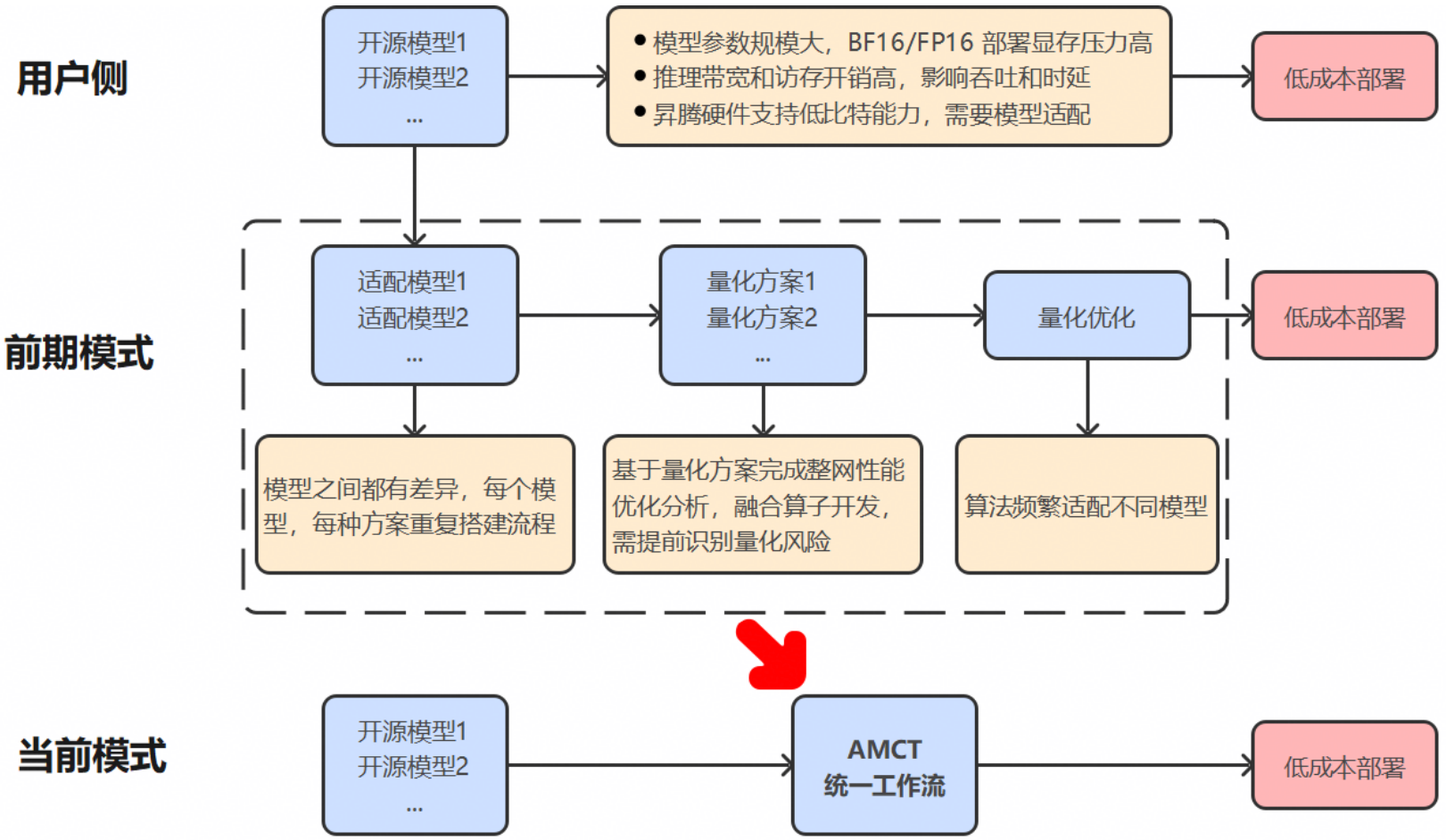
Part 1 CANNBot-Tilelang算子智能体

Part 2 AMCT低精度量化优化智能体

Part 3 CANNBot-模型部署优化智能体

模型 Agent 工作流 —— AMCT模型量化

设计背景：



简化整体流程，避免复杂繁琐重复的工作，搭建一体式工作流

CANN

模型 Agent 工作流 —— AMCT模型量化

现有能力总结：

20+
模型支持

7
低bit浮点格式

10+
算法

01 覆盖主流模型

支持 DeepSeek v3.2、DeepSeek v4、Qwen3、Qwen3-Next、 Qwen3.5/3.6、GLM 5.1/5.2、Longcat Flash/Next、HunYuan3.0等主流模型，覆盖Dense/MoE, Attention(GQA, MHA, MLA)等典型结构

02 量化格式

支持Atlas-A3 Pod/950PR/DT上的INT4/8、FP8、MXFP4/8、HiF8六种格式以及下一代的HiF4格式，快速验证同一模型在不同格式下的精度损失

03 多种量化算法

支持同一模型在不同算法上的精度表现，快速沉淀面向相同架构下的精度补偿recipe，目前已支持LAC、LWC、AutoRound、SVDQuant、GPTQ、Hadamard(可学习)、Flatquant、OmniQuant、SmoothQuant等针对权重 outlier、激活范围、舍入误差、块内分布等问题选择优化算法

Adapt
模型适配
›
Evaluate
精度评估
›
Quantize
PTQ 优化
›
Deploy
部署导出

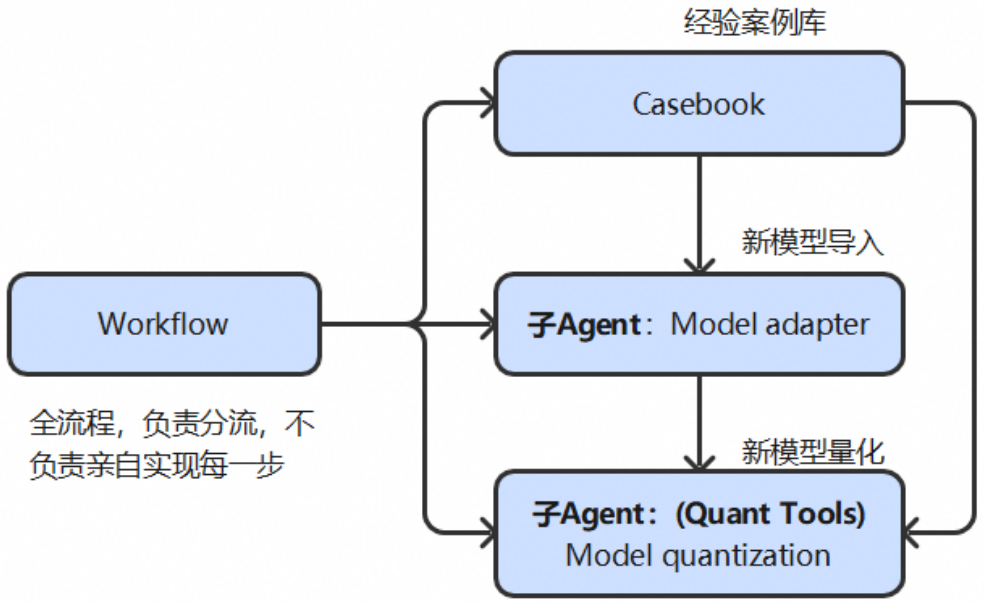
CANN

模型 Agent 工作流 —— AMCT模型量化

Agent 自动化量化权重和方案导出

整体流程

- **使用场景**：对原始权重, 希望得到昇腾可直接部署的量化权重；
- **方案选择**：检查可用性与是否已有参考方案，决定量化方式；
- **量化实施**：借助Quant Tools进行量化实验，导出可部署量化权重与方案介绍；
- **交付沉淀**：方案与轨迹更新为casebook，形成后续复用资产；

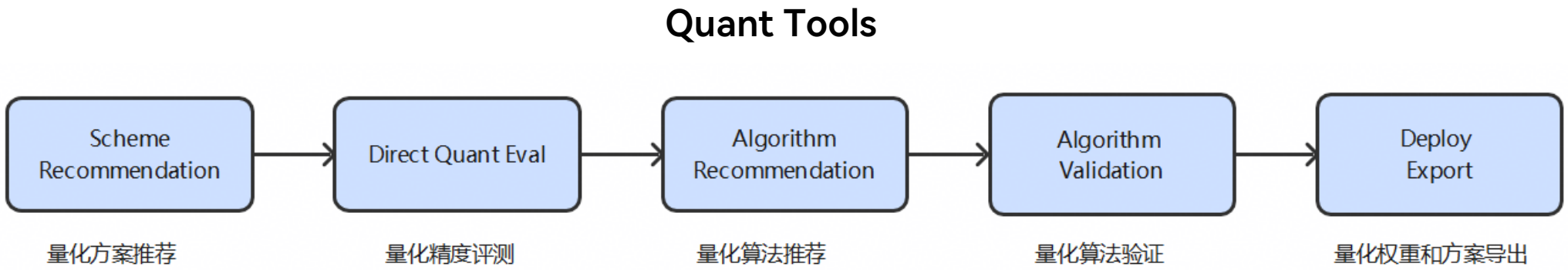


模块一介绍

- **Model adapter(只做新模型接入)**
  - 模型注册
  - BF16 基线测试与关闭量化后的浮点等价测试
  - 最小PTQ冒烟

模型 Agent 工作流 —— AMCT模型量化

模块二子Agent通过多skill组合，按照Casebook可灵活执行中间任意模块



模块二介绍

➢ **Model quantization(量化阶段总控)**

- 量化方案推荐：根据Casebook给出量化方案，或用户指定
- 直转量化评测：基于已有量化方案做直转量化测试
- 量化算法推荐及验证：根据Casebook推荐量化算法，并验证
- 量化权重导出：导出量化部署权重及方案说明文档

流程优势

➢ **易用性**

- 流程简单清晰
- 集合适配、量化、算法优化、导出，整体性强

➢ **节省上下文**

- Repo-Map：新模型/新算法导入无需全局扫描所有函数，对关键锚点文件扫描，节省上下文，同时了解工具整体框架

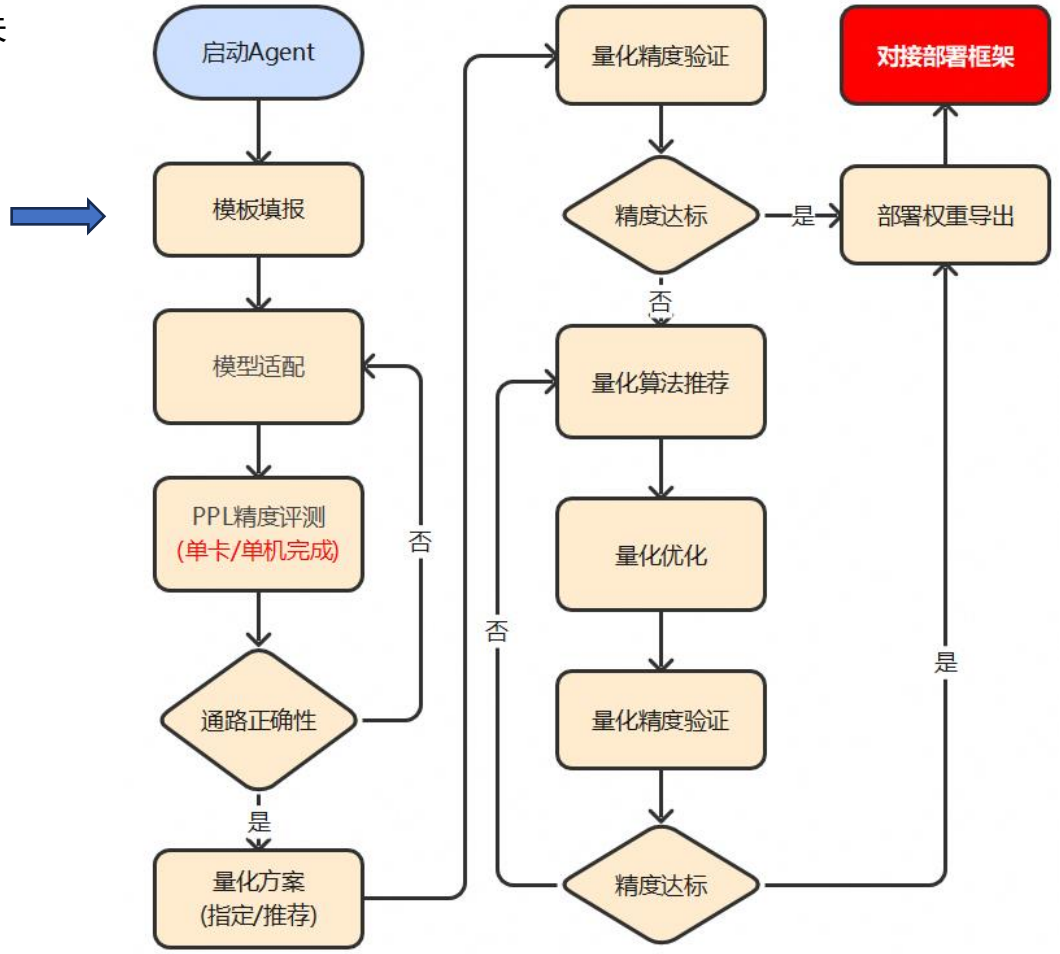
CANN

模型 Agent 工作流 —— AMCT模型量化

Agent模型量化优化全流程

通过“模板填报 + Agent 编排”的方式完成标准化闭环，用户只需补充关键信息，Agent 即可按模型状态自动推进下一步工作

```
- [必填] 模型权重路径：
- [必填] 期望的 `model_name`：
- [必填] 远程环境：
- [可选] 模型结构：dense / MoE / 其他
- [可选] 特殊结构：无 / MLA / linear attention / Mamba / 其他
- [可选] 当前阶段：未知 / 未适配 / 已适配未量化 / 已有量化结果 / 准备 deploy
- [可选] 目标：先打通 BF16 / 完成 model-adapter / 完成第一轮量化 / 一直到 deploy
- [可选] 是否允许复用已有结果：是 / 否
- [可选] 量化偏好：
  - `quant_target`：
  - `quant_dtype`：
  - `bits`：
  - `algos`：
- [可选] deploy 期望：
  - 是否要求最终导出权重：
  - 是否需要 infer 仓说明文档：
```



用户只需填报模板，Agent自动进行适配

包括**新数据格式**和**新算法**适配功能

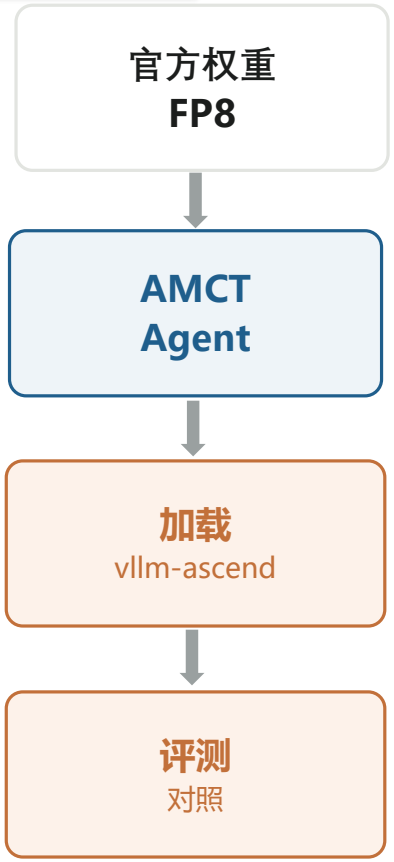
构建端到端自动化流程，将模型量化从分散的人工操作转变为可理解、可执行、可验证、可复用的工作流

CANN

模型 Agent 工作流 —— DSV4.1-Flash-HiFloat8实践

Agent基于AMCT离线量化落盘HiFloat8权重，到vLLM-ascend部署加载测评完整流程方案

整体流程



模块介绍

官方权重：下载准备官方权重

离线量化：AMCT Agent，兼容考虑落盘的数据格式weight(int8/uint8/fp8)和scale(int8/uint8/float32)，是否pack等操作，最终转换成HiF8，利用Uint8数据格式来落盘。

vLLM-ascend部署加载：基于官方vLLM版本，通过uint8数据格式透传HiFloat8数据格式，打上patch包，从而能够去加载转换好的离线权重，进行推理部署。

评测：利用AisBench/Evalscope评测框架

精度评测

数据集	BF16	HiF8	Dif
LiveCodeBench v5(pass@1)	67.07	67.37	+0.30
LongBench-v2	47.51	47.32	−0.19
MATH-500	96.80	97.20	+0.40
CMMLU(acc)	90.11	89.83	−0.28
MMLU-Pro(acc)	83.98	84.25	+0.27
HumanEval+(pass@1)	91.46	91.46	0.00
GPQA-Diamond	76.77	76.77	0.00

HiF8结果可以对齐BF16，**掉点在1个点以内**，属于合理范围内，流程无误。

CANN

目录

Part 1 CANNBot-Tilelang算子智能体

Part 2 AMCT低精度量化优化智能体

Part 3 CANNBot-模型部署优化智能体

背景：通用 Agent 进行NPU 模型优化的典型问题

➢ 通用 Agent 难以稳定完成长链路、强依赖、重验证的 NPU 推理优化任务

仓库经验难复用，阶段选型不准确

真实优化同类参考案例实现、推荐路径与踩坑经验，通用Agent缺少支撑

领域知识不完整，难把握优化边界

Agent不了解各种原子化CANN 优化能力的使用前提、组合关系与工程约束

验证门禁弱，难以稳定闭环

完整优化链路缺少逐阶段验证、异常回退和问题定位排查机制，难稳定闭环

缺少分层上下文与结构化流程支撑

长流程推理优化中上下文丢失，关键决策无法稳定传递，信息失焦容易局部反复试错

➢ 解决路径：把优化经验沉淀为可执行、可验证、可复用的 Skill 工作流

Agent 体系 = 领域知识（Skill） + Harness约束（上下文架构 / 专业化分工 / 持久化记忆 / 结构化执行）

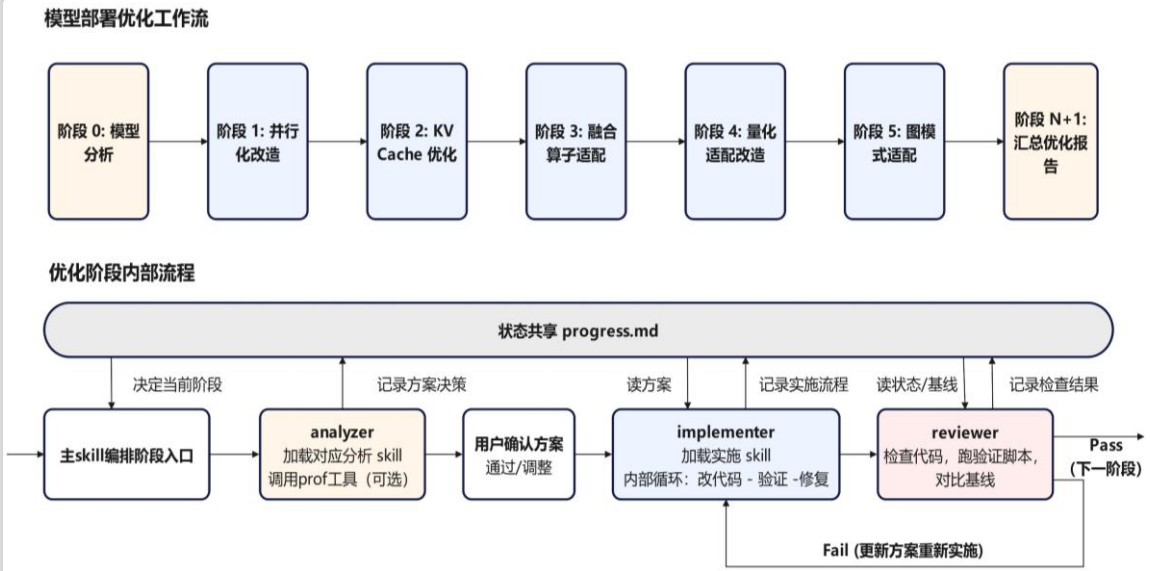
让 Agent 在 NPU 推理优化这种长链路任务上稳定产出，提升工作效率，降低使用门槛

模型 Agent —— 推理部署优化

模型基础部署优化

➢ Agent 端到端工作流 —— 稳定实现基础部署优化闭环

- 覆盖模型迁移适配、并行切分、KVCache 优化、融合算子替换、量化、图模式适配、debug 调试等 CANN 原子化优化特性，支持新模型一键式NPU部署优化。
- 通过完善的Harness 约束机制，包括 Agent team 角色分工、Progress 状态同步、Hook 约束和验证门禁，确保 Agent 稳定高效完成优化流程。



➢ 专业化分工：Agent Team

角色	职责	权限
Workflow（主流程编排）	编排 / 派发 / 用户确认 / 进度管理	不直接改代码，只做流程编排
Analyzer（分析）	架构分析 / 方案设计	只读代码，只写 progress.md
Implementer（实施）	代码改造 / 调试 / 必须自验证	读写全部文件
Reviewer（验证）	精度 / 性能验证 / 写报告	禁改模型实现代码

➢ 结构化执行：Hook 机制

软约束（Skill 文字规则、CLAUDE.md）：建议性，Agent 可能不遵守

硬约束（Hook 脚本）：确定性，工具调用前后强制执行，无法绕过

➢ 持久化记忆：状态管理

progress.md 记录模型分析、优化方案与各阶段进展，可承接既有进度，subagent 之间也通过它交接上下文。

模型 Agent ——进阶性能调优

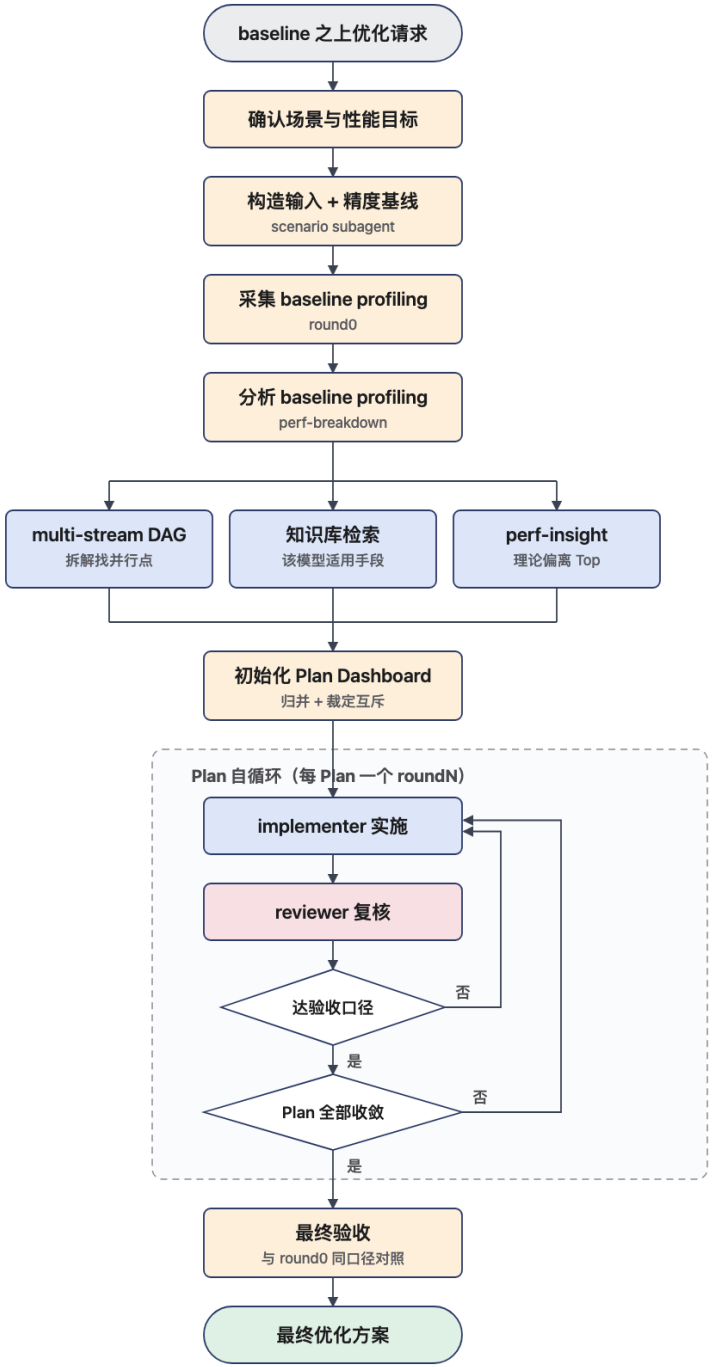
➢ 工作流驱动的人机协同 —— 进阶优化追求极致性能

- 通过profiling 采集与性能分析拆解 Skill，支持拆分模块/层耗时与抖动等性能瓶颈识别呈现，为优化项提供证据支撑。
- 利用多流、SuperKernel、Prefetch 等进阶优化 Skill， 使能优化特性进一步提升推理性能。

➢ Plan 自循环机制

- 在优化项发现给出一批待验证的优化方向，Plan 循环负责将它们逐个实施、验证、保留或淘汰，并在过程中派生新方向，直至收敛。

状态	含义
待实现	尚未实施，或证据不足但仍有继续实施、调测的价值。
通过	有明确收益或必要价值，且功能、精度、回退路径和副作用都已检查通过。
淘汰	无收益、风险过高、实现失败、破坏功能或精度，或被同一互斥组里更优的 Plan 替代。



模型 Agent ——部署基线与并行策略

➢ 阶段 0：模型分析与基线建立

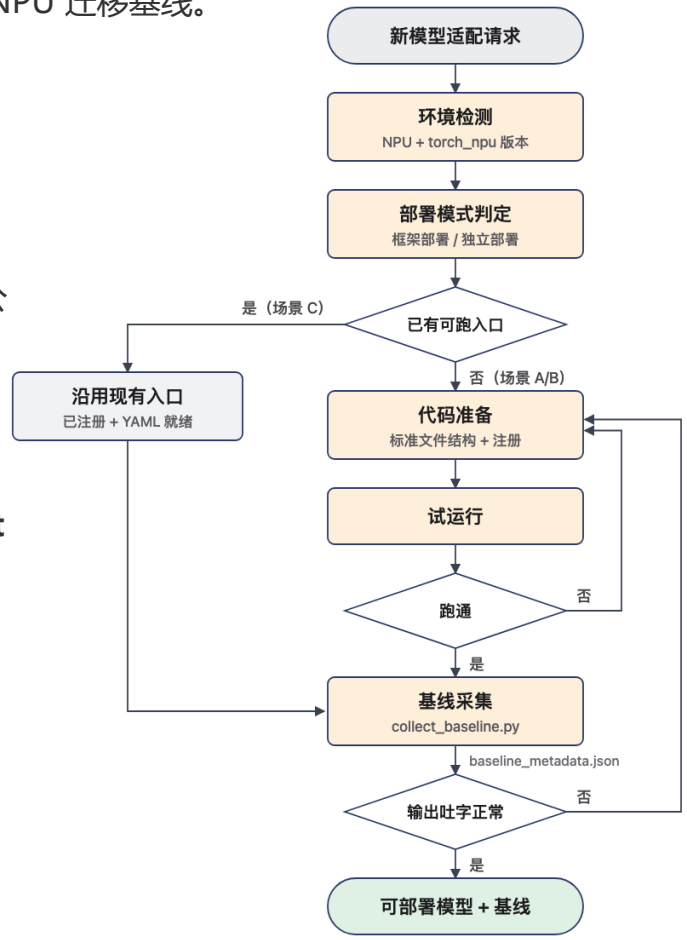
Agent 分析与实施：参考 **model-infer-migrator**，进行信息收集（模型目录 / HF链接 / 权重路径 / 部署模式等），基于模型源码与 config.json 识别架构，对部署方案初评估，建立 NPU 迁移基线。

部署模式确认

Recipes 仓库框架部署（推荐）：注册 support_models.py，公共流程由 executor/core/ 统一

独立部署：Runner 自管推理循环，适用仓外项目

判定优先级：**dispatch prompt 显式指定 > 用户对话指定 > 工作目录自检**



➢ 阶段 1： 并行切分改造

Agent 分析与实施：参考 **model-infer-parallel-analysis / -impl**，先分析模型结构与硬件规格，对照最佳实践部署样例，分析计算、显存、通信，确定整体并行策略；再落地切分、搭建各通信组、按维加载权重，并验证多卡输出语义正确。Agent 实现基础部署方案，人机协同优化通信方式追求极致性能。

第一层 架构主干

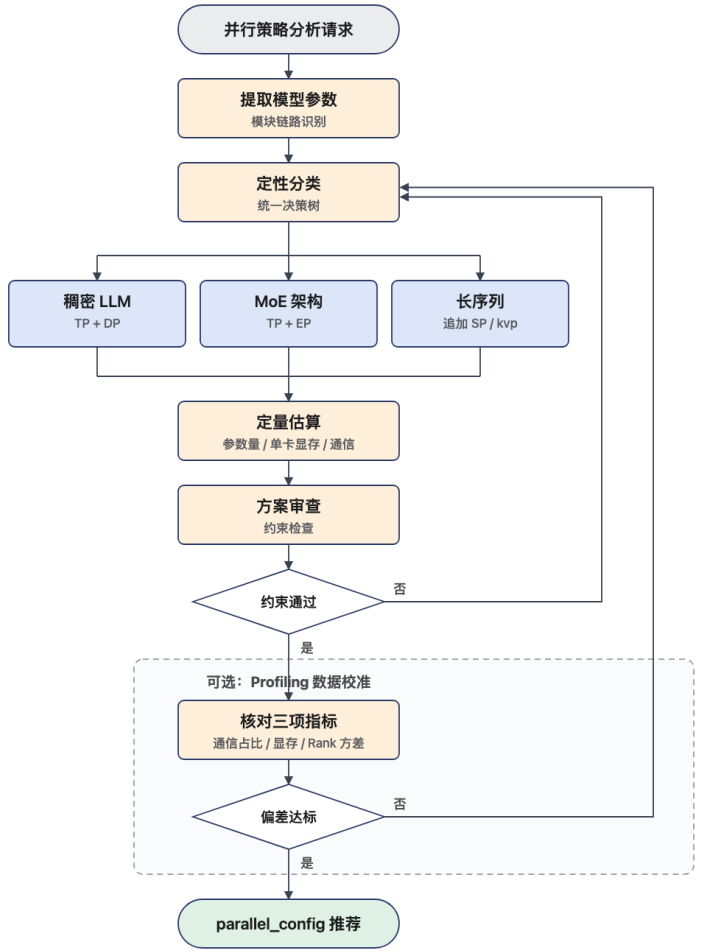
例：Dense 走 DP/TP | MoE 比较 EP 和 TP | MLA 独立评估 oproj_tp | 大词表 Embed/LMHead 独立设 TP

第二层 场景调整

例：高吞吐大 batch → attn_tp=1 最大化 DP | 低时延小 batch → 增大 attn_tp 并行度

第三层 定量分析

例：分析方案可行性，analyzer 输出 2-3 个候选 parallel_config，用户从候选里选择

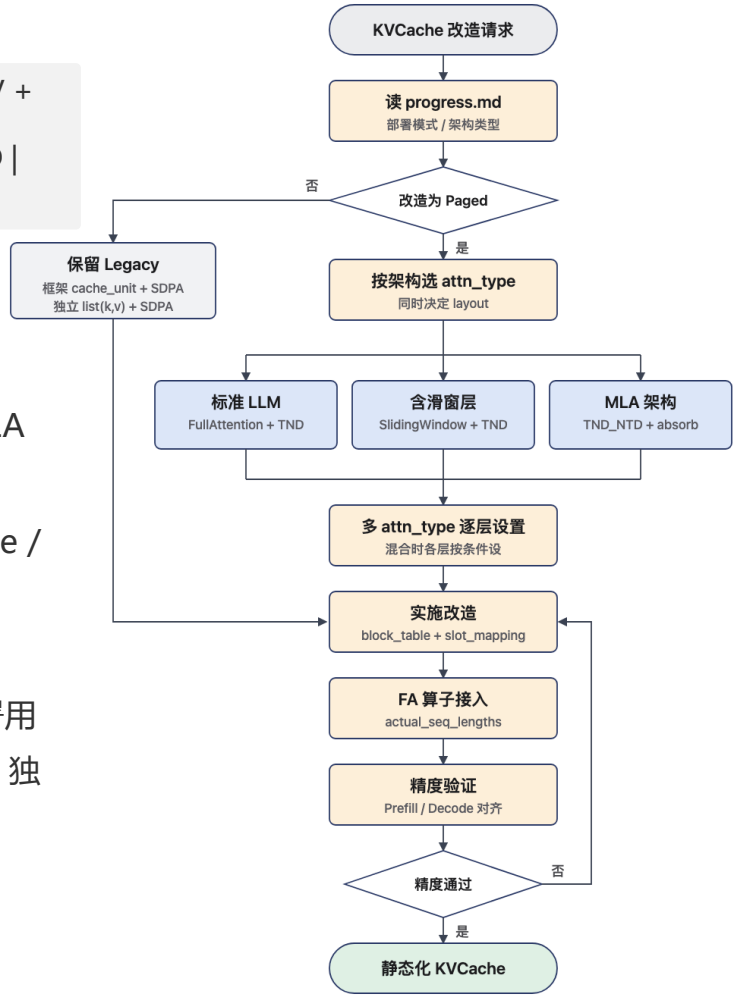


模型 Agent ——Cache 改造与融合算子接入

➢ 阶段 2：KVCache + FA 改造

Agent 分析与实施：参考 **model-infer-kvcache**，把 KV 管理从模型自管缓存改造为框架公共分页管理，接入 FA 融合算子，构造 block_table 与 slot_mapping。

migrator 过渡骨架（Legacy KV + HF SDPA + BSH）
→ 标准 LLM：**PA + FA + TND | MLA absorb**



① **快速选型**：Paged 模式 / attn_type / FA v1 vs v2 / MLA absorb

② **核心数据结构**：block_table / slot_mapping / actual_seq_lengths / kv_len

③ **按部署模式分流**：框架部署用 cache_entries + BlockPool；独立部署 Runner 自管

➢ 阶段 3： 融合算子接入

Agent 分析与实施：参考 **model-infer-fusion**，**按模块**识别Attention、MOE、Norm等可融合的计算模式，按计算语义匹配并替换为 CANN 已支持的 torch_npu / AscendC 融合算子，核对入参选择与前置改造项，分组接入并验证与原实现对齐。

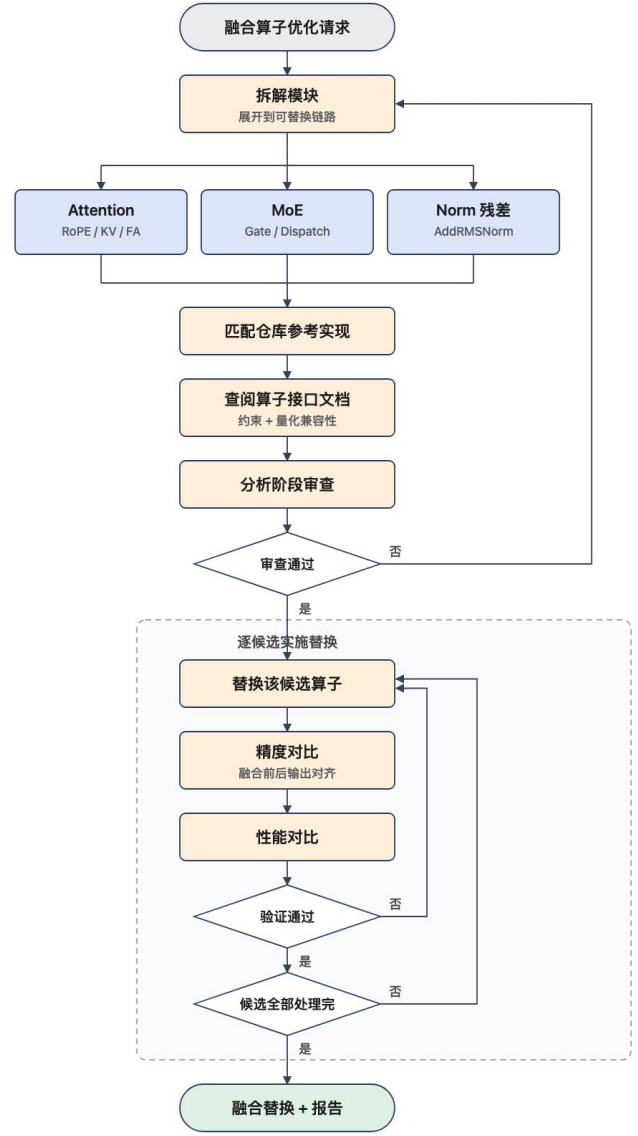
① **模块拆解**

② **仓库匹配**

③ **API 文档校验**

④ **逐模块替换**

⑤ **清单审查校验**



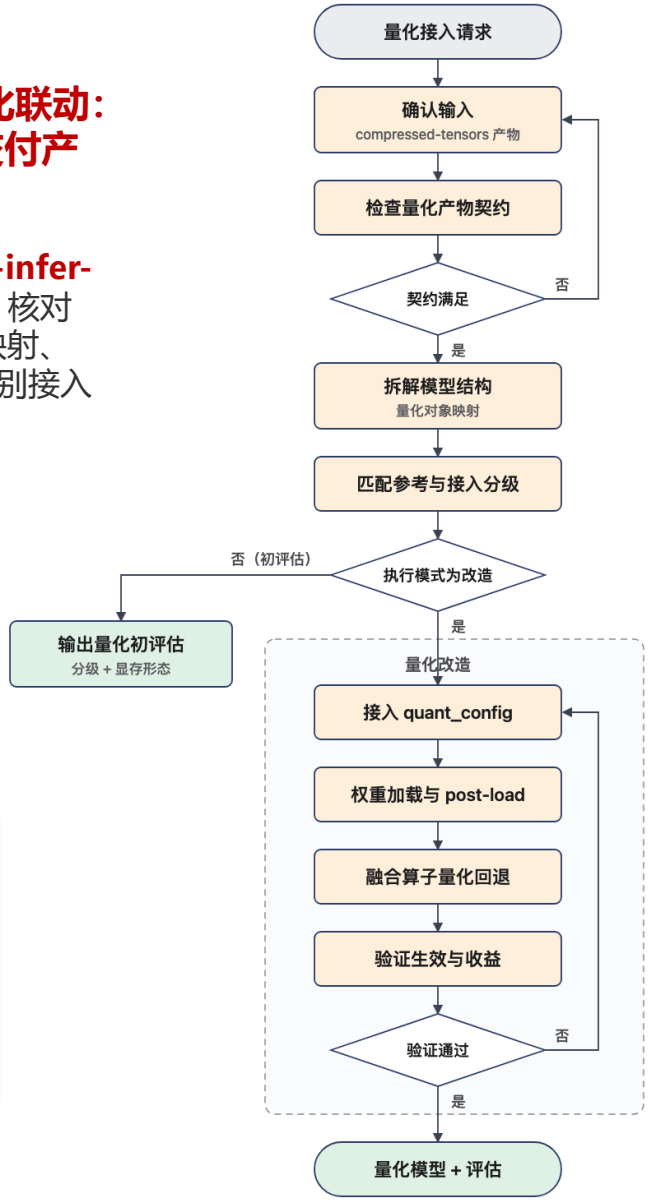
当前新算子开发/接入仍需部分人工指导，未来将联动算子生成 agent，实现从自动融合范围分析到新算子开发的完整 Agent 流程。

模型 Agent ——量化改造与图模式适配

➢ 阶段 4：量化改造

AMCT 量化仓与推理部署优化联动：生成量化权重/方案 → 对接交付产物 → 模型量化接入

Agent 分析与实施：参考 **model-infer-quantization**，先做量化初评估、核对量化产物契约，再建立 runtime 映射、按参数属性加载量化分片进去，识别接入量化算子。



契约检查：

检查 quantization_config / quant_method / config_groups 等

任何关键项缺失 → 直接输出补充诉求

融合算子兼容性分级

A 量化专用算子

B 可接受量化输入

C 需量化版本替换

D 仅特定契约支持量化

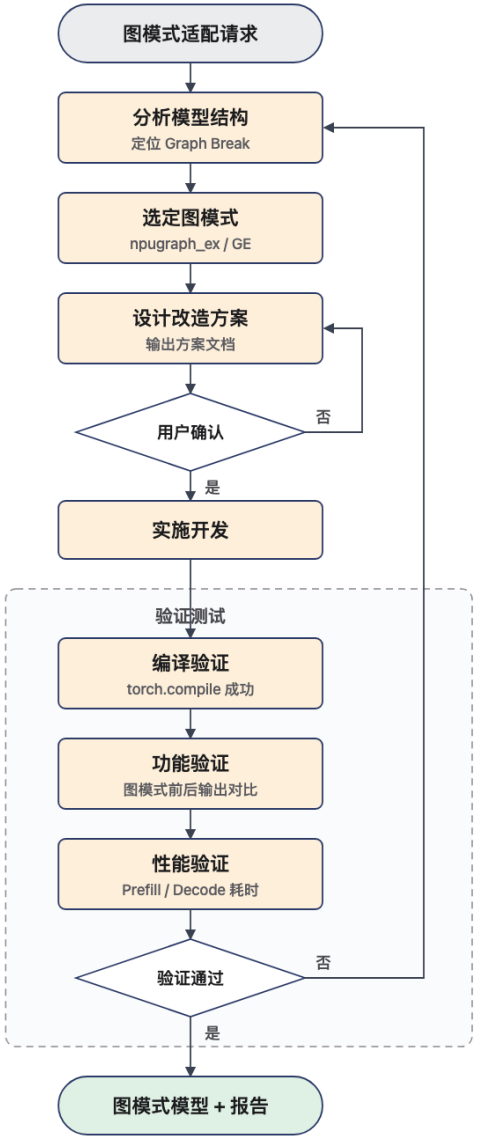
https://gitcode.com/cann

➢ 阶段 5： 图模式适配

Agent 分析与实施：参考 **model-infer-graph-mode**，把 decode 路径适配到图模式后端（950 PR/DT用 npugraph_ex），定位并消除 graph break，接入编译缓存降低重复编译开销。

npugraph_ex是一个基于CANN的PyTorch FX图优化组件，通过torch.compile入口无缝对接NPU图模式（ACLGraph），帮助开发者在零额外成本下充分释放NPU性能。

- **快速接入**：CANN提供ACLGraph，有效的消除了算子调度时延，并已经集成在PyTorch中，可快速使能；
- **高性能**：在复用原有ACLGraph调度能力前提下，借助于PyTorch的FX图，进一步叠加NPU亲和的优化；
- **可集成**：面向大模型推理的服务化框架，可以做到快速集成，无缝对接到vLLM/SGLang等服务化框架中。



使用入口与优化效果

入口一：cann-recipes-infer 仓库直接使用

Skill / Agent / Hook 已内置在仓库

一键部署：bash scripts/init-agent.sh

平台适配： claude / opencode / codex

入口二：CANNBot

装到任意已有项目， plugin 形式安装

安装方式：Plugin Marketplace / init.sh 脚本 / 手动安装

平台适配：Claude Code / OpenCode / TRAE / Cursor

启动提示词

全流程 「帮我优化 XX 模型」

→ model-infer-optimize 入口启动，完整优化阶段

单阶段 「分析 XX 模型的并行策略」

→ 单独调用某 skill（kvcache / fusion 等）

模型 Agent 优化效果样例

➢ **Hy3 模型基础部署优化各阶段性能/耗时统计**

部署场景：昇腾 950 PR/DT 4 die

优化阶段	工作耗时	Decode 单步	相对基线
模型分析+部署骨架	~0.7 hr	-	-
AMCT量化方案	~0.5 hr	-	-
并行切分方案与实施	~2.2 hr	103.18 ms	1.0x
KVCache 与 FA 改造	~2.7 hr	96.40 ms	1.07x
融合算子适配	~2.5 hr	44.94 ms	2.30x
MXFP 量化接入	~1.2 hr	42.59 ms	2.42x
图模式适配	~1.0 hr	17.98 ms	5.74x
合计	**~10.8 hr**	**17.98 ms**	**5.74x**

80% 方案设计+代码开发可由 Agent 完成

模型 Agent —— 仓库链接与样例

仓库样例

cann-recipes-infer 仓库已开源代码与技术文档

Agentic 全流程覆盖的部署优化样例：

Hunyuan3	Longcat-Flash-Lite
Gemma-4	Step3.7-Flash

Agentic 深度协同的高性能推理样例：

Deepseek V4.1 Flash	Kimi-K3

cann-recipes-infer 代码仓

CANNBot 代码仓

AMCT代码仓

cann-recipes-infer:
https://gitcode.com/cann/cann-recipes-infer

CANNBot:
https://gitcode.com/cann/cannbot-skills

AMCT:
https://gitcode.com/cann/amct

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann